Name: T.MOHANA

Hall Ticket: 2303A52446

Batch: 41

Task 1: Movie Database API Integration

Prompt:

Generate a Python script using the requests library to fetch movie details from the OMDb API by title. Include error handling for invalid movie names, API key issues, timeouts, and empty responses.

Code:

```
#Generate a Python script using the requests library to fetch movie details from the OMDb
API by title. Include error handling for invalid movie names, API key issues, timeouts,
and empty responses.
import requests

def fetch_movie_details(movie_title, api_key):
    url = f"http://www.omdbapi.com/?t={movie_title}&apikey={api_key}"

    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()  # Raise an error for HTTP errors
    except requests.RequestException as e:
        print(f"Error fetching movie details: {e}")
        return None

    data = response.json()

    if data.get("Response") == "False":
        print(f"Movie not found: {movie_title}")
        return None

    return data
# Example usage
api_key = "a28f0b2c"
movie_title = input("Enter the movie title: ")
movie_details = fetch_movie_details(movie_title, api_key)
if movie_details:
    print(f"Title: {movie_details.get('Title')}")
    print(f"Year: {movie_details.get('Year')}")
    print(f"Genre: {movie_details.get('Genre')}")
    print(f"Director: {movie_details.get('Director')}")
    print(f"Plot: {movie_details.get('Plot')}")
```

Output:

```
PS C:\Users\lilac\New folder\fashion-store\server> & C:\Users\lilac\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/lilac/New folder/fashion-store/
erver/Untitled-2.py"
Enter the movie title:  Guardians of the Galaxy Vol. 2
Title: Bonus Round: The Making of 'Guardians of the Galaxy Vol. 2'
Year: 2017
Genre: Documentary, Short
Director: N/A
Plot: A four-part documentary: In the Director's Chair with James Gunn (8:36): A look at the first film's success, ideas and visions for the second film, Gunn's
dedication and abilities, love for...
PS C:\Users\lilac\New folder\fashion-store\server>
```

Explanation:

This script uses the requests library to fetch movie data from the OMDb API and handles multiple failure scenarios gracefully. It ensures the program doesn't crash by catching errors like invalid input, API issues, and network failures.

Task 2: **Public Transport Schedule API**

Prompt:

Generate a Python script using requests to fetch public transport arrival times by stop ID with retry logic and error handling. Display the next 3 arrivals and handle invalid input, timeouts, and server errors gracefully.

Code:

```python
import requests
import time

# Real working API (London Transport)
BASE_URL = "https://api.tfl.gov.uk/StopPoint/"

def fetch_arrival_times(stop_id):
    url = f"{BASE_URL}{stop_id}/Arrivals"
    retries = 2  # minimum 1 retry

    for attempt in range(retries + 1):
        try:
            response = requests.get(url, timeout=5)
            response.raise_for_status()

            data = response.json()

            # Handle empty response
            if not data:
                print("❌ No arrivals found. Check Stop ID.")
                return None

            # Sort by arrival time
            data.sort(key=lambda x: x.get("timeToStation", 0))

            # Extract next 3 arrivals
            arrivals = []
            for item in data[:3]:
                arrivals.append({
                    "line": item.get("lineName", "N/A"),
                    "destination": item.get("destinationName", "N/A"),
                    "time": item.get("timeToStation", 0) // 60
                })

            return arrivals
```

```python
        except requests.exceptions.Timeout:
            print("⚠ Request timed out. Retrying...")

        except requests.exceptions.ConnectionError:
            print("❌ Server unavailable / Network issue.")
            return None

        except requests.exceptions.HTTPError:
            if response.status_code == 404:
                print("❌ Invalid Stop ID.")
            else:
                print("❌ HTTP error occurred.")
            return None

        except Exception as err:
            print(f"❌ Unexpected error: {err}")
            return None

        time.sleep(2)

    print("❌ Failed after retry attempts.")
    return None


# ------------ MAIN ------------
if __name__ == "__main__":
    stop_id = input("Enter Stop ID: ").strip()

    # Better validation (not just digits)
    if not stop_id:
        print("❌ Stop ID cannot be empty.")
    else:
        arrivals = fetch_arrival_times(stop_id)

        if arrivals:
            print("\n🚌 Next 3 Arrivals")
            print("-" * 30)
            for i, a in enumerate(arrivals, 1):
                print(f"{i}. Line {a['line']} → {a['destination']} in {a['time']} min")
        else:
            print("⚠ Could not retrieve arrival data.")
```

Output:

```
Command executed 39 secs ago and took 23.494s
PS C:\Users\lilac\New folder\fashion-store\server> & C:\Users\lilac\AppData\Local\Programs\Python\Python313\python.
erver/Untitled-2.py"
Enter Stop ID: 490008660N

🚌 Next 3 Arrivals
------------------------------
1. Line 214 → Highgate Village in 5 min
2. Line 88 → Parliament Hill Fields in 7 min
3. Line 214 → Highgate Village in 14 min
PS C:\Users\lilac\New folder\fashion-store\server>
⊗ 0 ⚠ 0    Update is ready, click to restart.    Ln 2, Col 1 (2319
```

Explanation:

This script fetches transport arrival times using an external API, sorts them, and displays the next three arrivals. It includes retry logic and robust error handling to prevent crashes due to invalid input, network issues, or server errors.

Task 3: Stock Market Data API

Prompt:

Generate a Python script using requests to fetch stock data from a public financial API (e.g., Alpha Vantage or Yahoo Finance) using a user-input stock symbol. The script must safely parse JSON, validate fields, handle invalid symbols, API rate limits, missing data, and network errors, and display formatted output for price, high, and low.

Code :

```
#Generate a Python script using requests to fetch stock data from a public financial API
(e.g., Alpha Vantage or Yahoo Finance) using a user-input stock symbol. The script must
safely parse JSON, validate fields, handle invalid symbols, API rate limits, missing data,
and network errors, and display formatted output for price, high, and low.
import requests

API_KEY = "ZKU956STHE1S2YKU"

symbol = input("Enter stock symbol: ").upper()

url =
f"https://www.alphavantage.co/query?function=GLOBAL_QUOTE&symbol={symbol}&apikey={API_KEY}
"

try:
    response = requests.get(url, timeout=5)
    data = response.json()

    if "Global Quote" not in data or not data["Global Quote"]:
        print(" Invalid stock symbol or no data.")
    elif "Note" in data:
        print(" API rate limit exceeded. Try later.")
    else:
        quote = data["Global Quote"]

        print("\n Stock Details")
        print(f"Symbol : {quote.get('01. symbol')}")
        print(f"Price  : {quote.get('05. price')}")
        print(f"High   : {quote.get('03. high')}")
        print(f"Low    : {quote.get('04. low')}")

except Exception as e:
    print(" Error:", e)
```

Output:

```
erver/Untitled-2.py"
Enter stock symbol: MSFT

 Stock Details
Symbol : MSFT
Price  : 373.4600
High   : 373.6400
Low    : 364.1500
```

Explanation:

The script fetches live stock data from an API using a user-provided symbol and safely parses the JSON response to display price, high, and low values. It handles errors like invalid symbols, API limits, and network issues to prevent crashes and provide clear messages.

Task 4: Geolocation API (IP Address Lookup)

Prompt:

Generate a Python script using the requests library to fetch geolocation details (country, city, ISP) for a given IP address using a public API. Include IP format validation and error handling for invalid input, API failures, and JSON decoding errors.

Code:

```python
import requests
import re

def is_valid_ip(ip):
    # Simple IPv4 validation
    pattern = r"^(?:[0-9]{1,3}\.){3}[0-9]{1,3}$"
    return re.match(pattern, ip)


def get_location(ip):
    url = f"http://ip-api.com/json/{ip}"

    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()

        try:
            data = response.json()
        except ValueError:
            print(" Error: Invalid JSON response.")
            return

        # API failure
        if data.get("status") != "success":
            print(" Error: Invalid IP or API failed.")
            return

        print("\n Geolocation Details")
        print("-" * 30)
```

```python
        print(f"Country : {data.get('country')}")
        print(f"City    : {data.get('city')}")
        print(f"ISP     : {data.get('isp')}")

    except requests.exceptions.Timeout:
        print(" Error: Request timed out.")

    except requests.exceptions.ConnectionError:
        print(" Error: Network problem.")

    except requests.exceptions.HTTPError:
        print(" Error: HTTP issue.")

    except Exception as e:
        print(f" Unexpected error: {e}")


# ------------ MAIN ------------
if __name__ == "__main__":
    ip = input("Enter IP address: ").strip()

    if not ip:
        print(" Error: IP cannot be empty.")

    elif not is_valid_ip(ip):
        print(" Error: Invalid IP format.")

    else:
        get_location(ip)
```

Output:

```
erver/Untitled-2.py"
Enter IP address: 8.8.8.8

 Geolocation Details
 -------------------------------
Country : United States
City    : Ashburn
ISP     : Google LLC
PS C:\Users\lilac\New folder\fashion-store\server>
0 ⚠ 0        Update is ready, click to restart.
```

Explanation:

This script validates the IP address format and fetches geolocation details from a public API.
It handles errors like invalid input, API failures, and JSON parsing issues to ensure stable execution.

Task 5: Real-Time Application: Payment Gateway Simulation API

Prompt:

Generate a Python script using requests to simulate payment processing via a sandbox API with POST requests.
Include input validation, error handling (400/401/500, network issues), and logging of failed transactions to a file.

Code:

```python
import requests
import json
from datetime import datetime

# ------------ VALIDATION ------------
def validate_input(amount, currency, method):
    if amount <= 0:
        return "Amount must be greater than 0"

    if currency not in ["USD", "INR", "EUR"]:
        return "Unsupported currency"

    if method not in ["card", "upi", "netbanking"]:
        return "Invalid payment method"

    return None


# ------------ LOGGING ------------
def log_failure(data, error):
    with open("failed_transactions.log", "a") as file:
        file.write(f"{datetime.now()} | Data: {data} | Error: {error}\n")


# -------- PAYMENT FUNCTION --------
def process_payment(amount, currency, method):
    url = "https://httpbin.org/post"

    payload = {
        "amount": amount,
        "currency": currency,
        "method": method
    }

    error = validate_input(amount, currency, method)
    if error:
        print(f" Validation Error: {error}")
        log_failure(payload, error)
        return

    try:
        response = requests.post(url, json=payload, timeout=5)

        # Handle HTTP errors manually
        if response.status_code == 400:
            raise Exception("Bad Request (400)")
        elif response.status_code == 401:
            raise Exception("Unauthorized (401)")
        elif response.status_code == 500:
            raise Exception("Server Error (500)")

        data = response.json()
```

```python
        print("\n Payment Successful")
        print("-" * 30)
        print(f"Amount   : {amount}")
        print(f"Currency : {currency}")
        print(f"Method   : {method}")
        print(f"Status   : SUCCESS")

    except requests.exceptions.Timeout:
        print(" Error: Request timed out.")
        log_failure(payload, "Timeout")

    except requests.exceptions.ConnectionError:
        print(" Error: Network issue.")
        log_failure(payload, "Connection Error")

    except Exception as e:
        print(f" Payment Failed: {e}")
        log_failure(payload, str(e))


# ------------ MAIN ------------
if __name___ == "__main__":
    try:
        amount = float(input("Enter amount: "))
        currency = input("Enter currency (USD/INR/EUR): ").upper()
        method = input("Enter payment method (card/upi/netbanking): ").lower()

        process_payment(amount, currency, method)

    except ValueError:
        print(" Invalid input. Amount must be a number.")
```

Output:

```
PS C:\Users\lilac\New folder\fashion-store\server> & C:\Users\lilac\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/lilac/New folder/fashion-store/server/Untitled-2.py"
Enter amount: 30000
Enter currency (USD/INR/EUR): USD
Enter payment method (card/upi/netbanking): card

 Payment Successful
------------------------------
Amount   : 30000.0
Currency : USD
Method   : card
Status   : SUCCESS
PS C:\Users\lilac\New folder\fashion-store\server>
```

Explanation:

This script simulates payment processing using a POST request and validates user input before sending data. It handles API errors, network issues, and logs failed transactions to ensure reliability and traceability.